

Optimisation Par Colonie De Fourmi d'un Chemin Sur Un Paysage Fractal

1 Les algorithmes coopératifs

Le terme « Intelligence en essaim » a été introduit en 1989 par Beni et al. Cette classe d'algorithmes, souvent inspirés par le comportement des insectes sociaux, met en place une population d'agents simples interagissant et communiquant indirectement avec leur environnement. Ces algorithmes constituent une classe d'algorithmes massivement parallèles pour résoudre une tâche donnée.

L'algorithme en essaim le plus connu est l'optimisation par colonies de fourmis (ACO) pour les problèmes combinatoires. Dans ce projet, on revient à l'inspiration originale des algorithmes ACO où une population d'agents simples (qui peuvent être vue comme imitant le comportement de fourmis réelles) résout efficacement le problème de fourragement (chercher le chemin le plus court de la fourmilière à une source de nourriture).

2 Modèle simple d'ACO

Dans cette partie, nous allons décrire un modèle simple de colonies de fourmis qui permet de résoudre le problème de fourragement. Il a été montré pour cet algorithme qu'il converge de façon sur-linéaire par rapport au nombre de fourmis.

2.1 Description du modèle

On considère un terrain représenté par une grille cartésienne 2D dont chaque cellule comporte une valeur donnant l'unité de temps pour la traverser (valeur comprise entre zéro et un). On considère un ensemble de m fourmis artificielles qui évoluent sur ces cellules et sur lesquelles elles mettent à jour des taux de phéromones $V_1(s)$ et $V_2(s)$. Chaque cellule s de la grille stocke deux valeurs réelles correspondant à deux types de phéromones :

- $V_1(s)$ indique aux autres fourmis le chemin d'exploration effectué ;
- $V_2(s)$ permet à la fourmi de retourner au nid lorsqu'elle a trouvé de la nourriture.

Sur la grille, on trouve quatre types de cellules : une cellule correspondant à la fourmilière, une cellule correspondant à la source de nourriture (qui pourrait ne pas être unique),

des cellules indésirables que les fourmis ne peuvent pas traverser, et le reste des cellules libres, c'est-à-dire explorables par les fourmis.

Une fourmi peut être dans deux états possibles : elle peut porter de la nourriture (état « chargée ») ou ne rien porter (état « non chargée »). En évoluant sur la grille, l'état des fourmis peut changer selon les règles naturelles suivantes :

- si une fourmi arrive sur une cellule contenant de la nourriture, son état devient « chargée » ;
- si une fourmi arrive à la fourmilière, son état devient « non chargée » ;
- quand une fourmi arrive à la fourmilière avec l'état chargée, un compteur d'unités de nourriture est incrémenté. Ce compteur servira d'indice de performance globale pour la population de fourmis choisie.

Décrivons la dynamique du modèle. Au départ :

- le compteur d'unités de nourriture est mis à zéro ;
- les m fourmis sont initialisées à des positions arbitraires (soit toutes dans la fourmilière soit initialisées uniformément sur la grille) ;
- toutes les fourmis sont non chargées ;
- les phéromones sont tous mis à zéro sur la grille.

À chaque pas de temps, une fourmi fait deux choses :

1. **Mise à jour des phéromones.** La fourmi met à jour les taux de phéromones $V_1(s)$ et $V_2(s)$ de la cellule sur laquelle elle se trouve en utilisant celles des quatre cellules voisines (on note $N(s)$ les voisins de s). La mise à jour requiert uniquement la connaissance du maximum et de la moyenne des cellules voisines :

$$\max_i(N(s)) = \max_{s' \in N(s)} V_i(s'), \quad \text{avg}_i(N(s)) = \frac{1}{4} \sum_{s' \in N(s)} V_i(s'),$$

où l'indice $i \in \{1, 2\}$ indique le type de phéromone considéré. La mise à jour se fait alors selon :

$$V_1(s) \leftarrow \begin{cases} 1, & \text{si la source de nourriture est en } s, \\ \alpha \max_1(N(s)) + (1 - \alpha) \text{avg}_1(N(s)), & \text{sinon,} \end{cases}$$

$$V_2(s) \leftarrow \begin{cases} 1, & \text{si la fourmilière est en } s, \\ \alpha \max_2(N(s)) + (1 - \alpha) \text{avg}_2(N(s)), & \text{sinon,} \end{cases}$$

avec $0 \leq \alpha \leq 1$.

2. **Déplacement.** La fourmi avance vers l'une de ses cellules voisines si elle n'a pas la valeur -1 :
 - avec une probabilité ε (taux d'exploration, $0 \leq \varepsilon \leq 1$), elle avance sur une cellule voisine choisie aléatoirement parmi ses quatre voisines ;

- avec une probabilité $1 - \varepsilon$, elle avance sur la cellule ayant le taux $V_1(s)$ le plus grand si elle n'est pas chargée, et sur la cellule ayant le taux $V_2(s)$ le plus grand si elle est chargée.

Le paramètre α est appelé paramètre de bruit. La fourmi avance dans le même pas de temps tant qu'elle n'a pas épuisé ses points de mouvement : à chaque pas de temps, la fourmi possède un taux de mouvement de 1 qu'elle doit dépenser en fonction de la valeur « unité de temps » du terrain qu'elle traverse (ce qui traduit la difficulté plus ou moins grande de traverser une zone). Sur la carte affichée à l'écran, plus une zone est claire, plus elle est difficile à traverser. À chaque pas de temps, les phéromones posés par les fourmis s'évaporent suivant un coefficient d'évaporation β :

$$V_i(s) \leftarrow \beta V_i(s),$$

avec β typiquement proche de 1.

Ainsi, pour l'instanciation du modèle, les paramètres suivants doivent être précisés :

- l'environnement : l'ensemble des cellules avec leur unité de temps, leur type (libre, indésirable, fourmilière ou nourriture) ;
- le nombre de fourmis m ;
- l'initialisation de la position des fourmis : soit toutes au départ dans la fourmilière soit réparties sur la grille uniformément ;
- les paramètres de bruit α , d'évaporation β et d'exploration ε .

Le modèle décrit ci-dessus est constitué d'agents réactifs simples qui communiquent indirectement au travers de l'environnement. Contrairement aux modèles classiques, il n'est pas nécessaire ici de conserver le trajet d'une fourmi par rapport à sa fourmilière grâce à l'utilisation d'un deuxième phéromone, ce qui rend nos agents complètement réactifs.

3 Création de l'environnement

Notre simulation nécessite l'utilisation d'une carte d'unités de temps traduisant la nature plus ou moins accidentée du terrain traversé par les fourmis (se traduisant par une valeur de temps que doit dépenser une fourmi pour rentrer dans une cellule). La génération d'une telle carte peut se faire soit de façon analytique (à l'aide d'une fonction) soit de façon stochastique en générant un plasma.

C'est la deuxième solution qui a été retenue pour ce projet. Un plasma est une carte générée de façon aléatoire où l'on s'assure que le gradient de valeurs entre deux points de la carte ne dépasse pas une certaine valeur d nommée déviation. On considère une grille composée de $n \times n$ sous-grilles. Ces sous-grilles ont un nombre de cellules par direction de $n_s = 2^k + 1$ cellules et se recouvrent avec les sous-grilles voisines à l'aide d'une rangée de cellules. L'algorithme est alors le suivant :

- on génère les coins de chaque sous-grille à l'aide d'un générateur pseudo-aléatoire dépendant d'une graine et de la coordonnée du sommet à évaluer : le premier coin de

- la première sous-grille est généré puis les autres coins sont obtenus séquentiellement à partir de ce premier coin en calculant une déviation inférieure à $d \times n_s$;
- on divise récursivement chaque sous-grille en quatre sous-grilles égales en calculant la valeur des cellules se trouvant au milieu de chaque bord et la valeur de la cellule se trouvant au centre de la sous-grille, en calculant des déviations inférieures à $d \times \frac{n_s}{2}$.
- Il faut veiller en permanence aux bords, communs à chaque sous-grille.

Une fois toutes les valeurs générées, on les normalise pour obtenir une carte de valeurs comprises entre zéro et un.

4 Mesure du temps passé par itération

Mesurer et noter le temps passé dans les différentes parties du code.

5 Vectorisation et parallélisation en mémoire partagée du code

5.1 Vectorisation

Dans un premier temps, on modifiera le code afin de vectoriser les algorithmes. Cela consiste à créer des tableaux contenant l'un les coordonnées de toutes les fourmis, le second leurs états, le troisième leurs graines, etc. Une fourmi sera ensuite repérée uniquement par son indice dans ces tableaux.

Mesurer de nouveau le temps passé dans les différentes parties du code et comparer avec la version d'origine.

5.2 Parallélisation en mémoire partagée

Paralléliser ensuite à l'aide d'OpenMP les boucles qui vous semblent adaptées et permettant une amélioration nette des performances du code.

Mesurer de nouveau le temps passé dans les différentes parties du code en fonction du nombre de cœurs de calcul utilisés et dresser un tableau donnant l'accélération obtenue en fonction du nombre de threads (et en fonction du nombre de cœurs de l'ordinateur sur lequel vous exécutez le code).

6 Parallélisation du code

Il existe deux façons de paralléliser le code (en fait trois, mais le troisième est beaucoup plus complexe à réaliser).

6.1 Première façon

L'idée est que chaque processus contienne l'environnement en entier et ne contrôle qu'une partie des fourmis tandis que, lors de la mise à jour des phéromones lors de la phase d'évaporation, chaque processus s'occupe d'une partie de la carte pour laquelle il calculera l'évaporation, à l'aide d'une parallélisation OpenMP si besoin.

Lorsque deux fourmis appartenant à deux processus différents se trouvent sur une même cellule, la valeur de la case va dépendre des valeurs des phéromones des cellules voisines, qui peuvent être différentes selon si d'autres fourmis locales sont passées avant ou non. Cependant, l'algorithme peut normalement prendre les fourmis dans un ordre arbitraire ; dans ce cas, on choisira de prendre la valeur la plus grande parmi tous les processus comme valeur de phéromone pour une cellule donnée.

Le problème de cette approche est qu'elle n'est efficace que si le nombre de fourmis est grand et la carte assez petite pour tenir en mémoire sur chaque processus. De plus, l'évaporation des phéromones se fait en parallèle, assurant ainsi un bon degré de parallélisme. Cette méthode assure un bon équilibrage des tâches, mais une grande quantité de données est échangée entre les processus.

Mesurer les temps pris par les différentes parties du code en parallèle en fonction du nombre de processus (en prenant un cœur par processus) et calculer l'accélération correspondante.

6.2 Seconde façon

Cette fois-ci, chaque processus prend en compte une partie de la carte et ne gère que les fourmis qui sont sur cette sous-carte. La difficulté est de gérer les bords de chaque sous-carte. De plus, il est possible qu'un processus n'ait pas de fourmis à gérer tandis qu'un autre en possède beaucoup (en particulier celui possédant la fourmilière). L'équilibre des tâches ne sera donc pas optimal, mais si le nombre de fourmis est assez grand ainsi que le coefficient d'exploration, il est probable que les fourmis soient relativement bien distribuées sur la carte.

L'avantage de cette méthode est que la mémoire occupée par l'application est bien plus petite qu'avec la première solution. De plus, il n'y a qu'un échange au bord des sous-cartes et donc peu de données échangées en définitive.

Indiquez votre stratégie pour programmer une telle parallélisation. **Bonus :** mettez en œuvre votre stratégie et calculez l'accélération en fonction du nombre de processus.